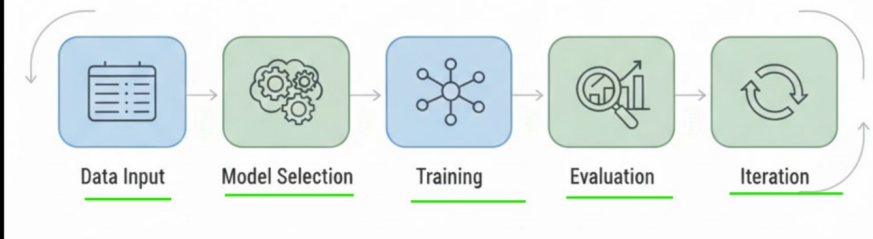
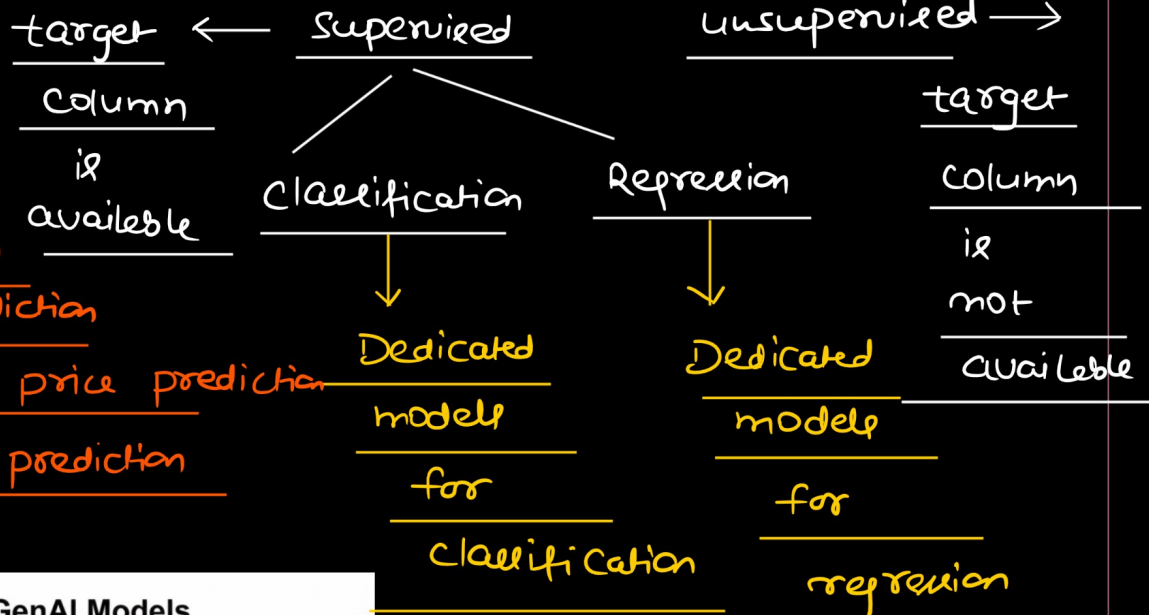


Model Development

Model Development and Evaluation Lifecycle



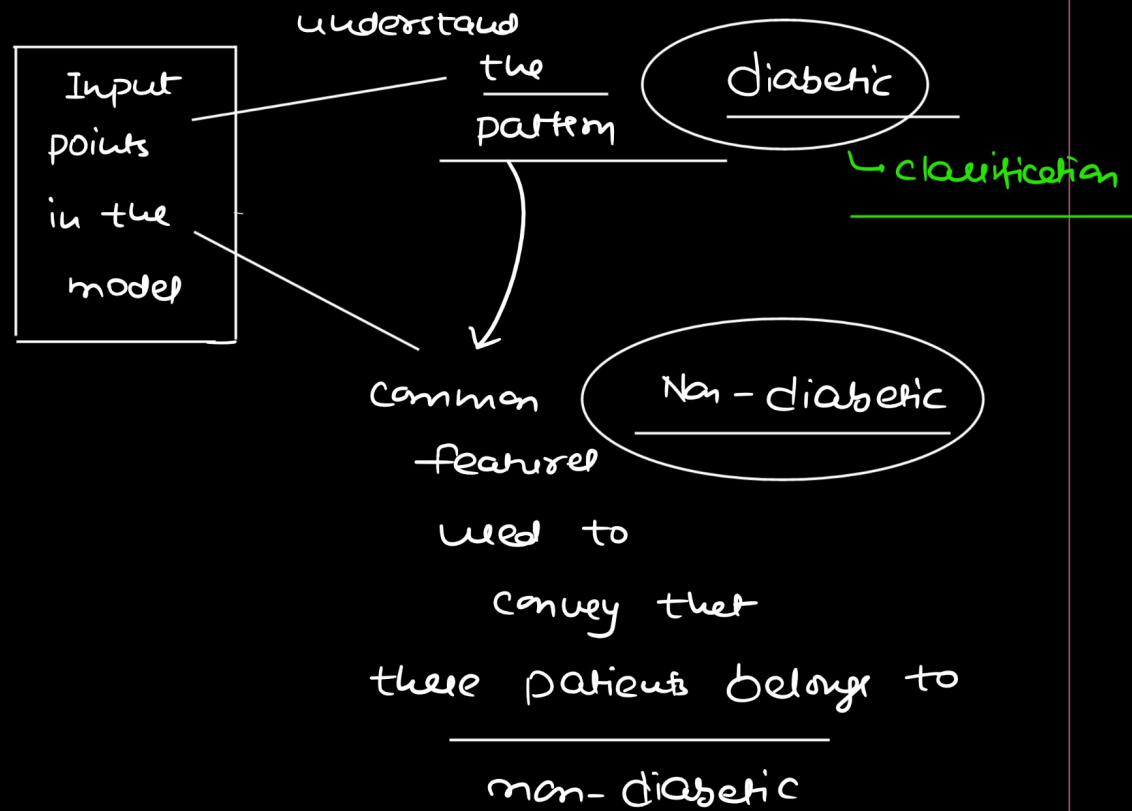
Machine Learning



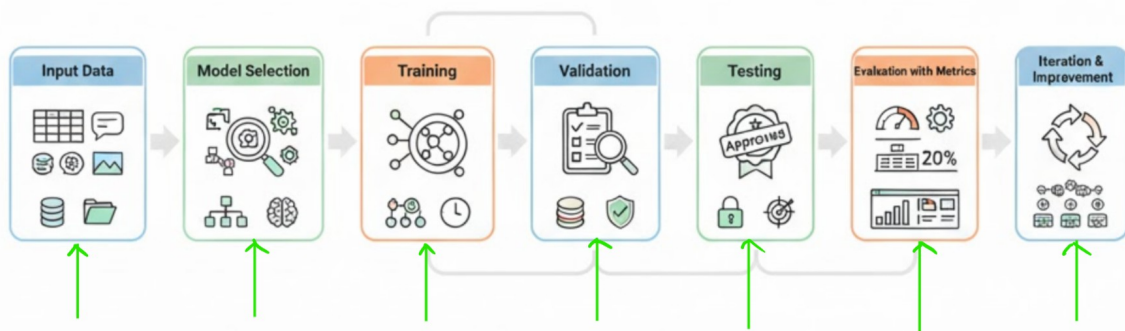
ML vs DL vs GenAI Models

Machine Learning (ML)	Deep Learning (DL) CNN	Generative AI (GenAI)
Works with structured data (tables). Finds simple patterns and relationships. Used for classification, prediction, clustering, forecasting.	Works in images, text, audio, video. Learns features automatically. Captures deeper patterns not easily designed by humans.	Creates text, ct, code, images, music. Understands, Learns from multiodal inputs. Learns for enornus datasets
Example Models Linear Regurssion, Logistic Regerssion, Decision Trees, Recision Trees, Random Forest, SVM	Example Models CNNs for images, RNNs / LSTms for sequences, Transformers or language and multihodal tasks	Example Models GPT-style LLMs, Diffusion image models, Multimoual models

- ① face recognition
- ② Image classification
- ③ Image xepmentation
- ④ object detection



Model Development Pipeline



Machine Learning Models

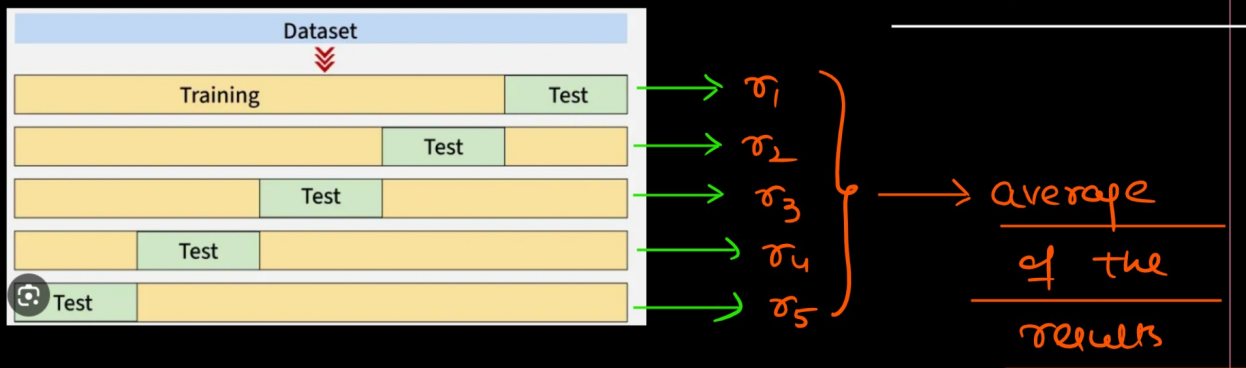
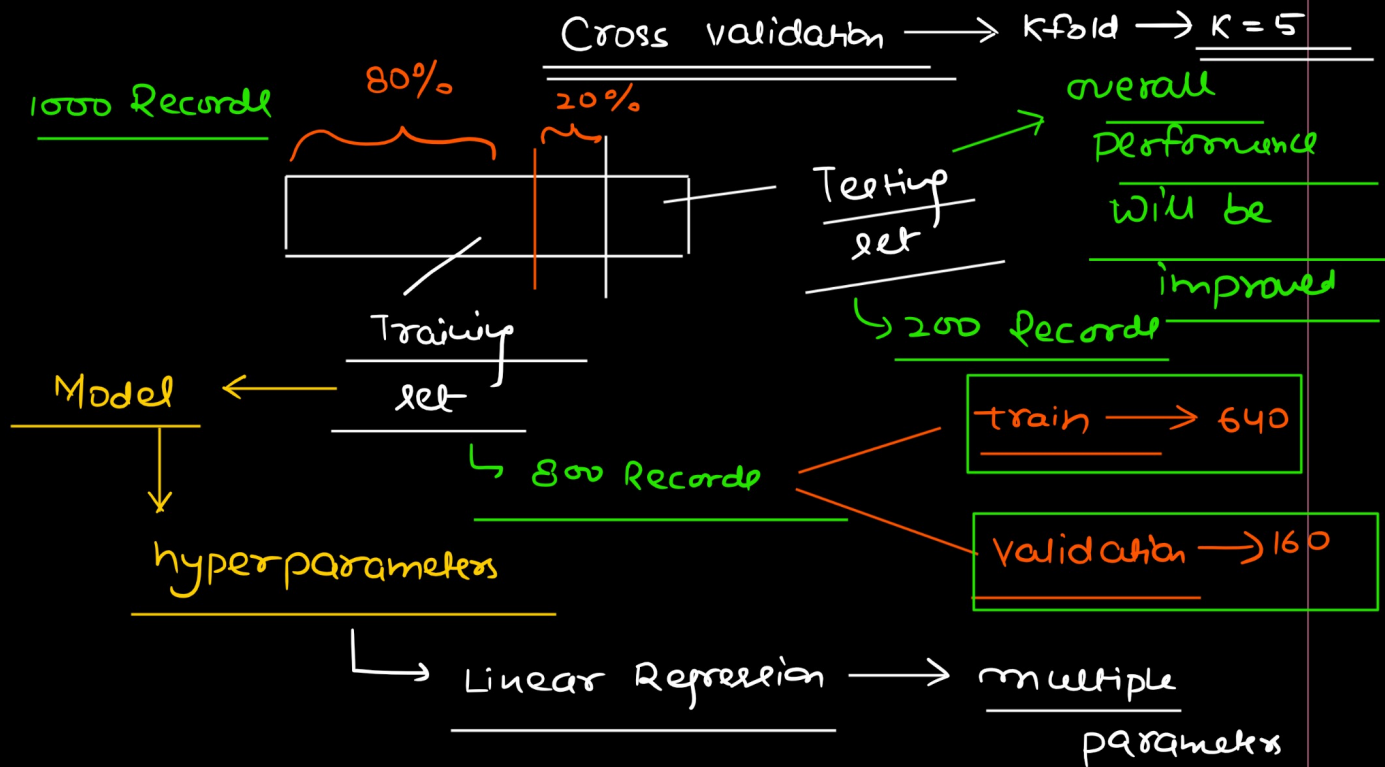
- **Linear Regression:** predicts continuous values like house price
- **Logistic Regression:** classifies two outcomes such as "spam or not"
- **Decision Trees:** easy-to-understand rules for classification
- **Random Forest:** stronger version of Decision Trees using many trees

Deep Learning Models

- **CNNs:** detect patterns in images (objects, faces, handwriting)
- **RNNs / LSTMs:** understand sequences (text, time series)
- **Transformers:** modern models for language, vision, and multimodal tasks

Generative AI Models

- **GPT-style models:** generate text, answer questions, summarize
- **Diffusion models:** create images (used in Stable Diffusion, Midjourney)
- **Multimodal models:** understand and generate multiple types of data



banking domain

Beginners often believe that deep learning is always the best choice. But experts know this is not true.

Simple models work better when:

the dataset is small

the patterns are straightforward

training speed matters

interpretability is important

the system must run on low-power devices

the project needs quick delivery

ML baseline model

A Practical Example: If a bank wants to classify customers into "high risk" and "low risk," a simple Logistic Regression model might perform extremely well. It will be:

fast to train

easy to understand

reliable

stable even with relatively small data

A complex neural network would take longer to train and would not offer better interpretability — which is important for finance and compliance.

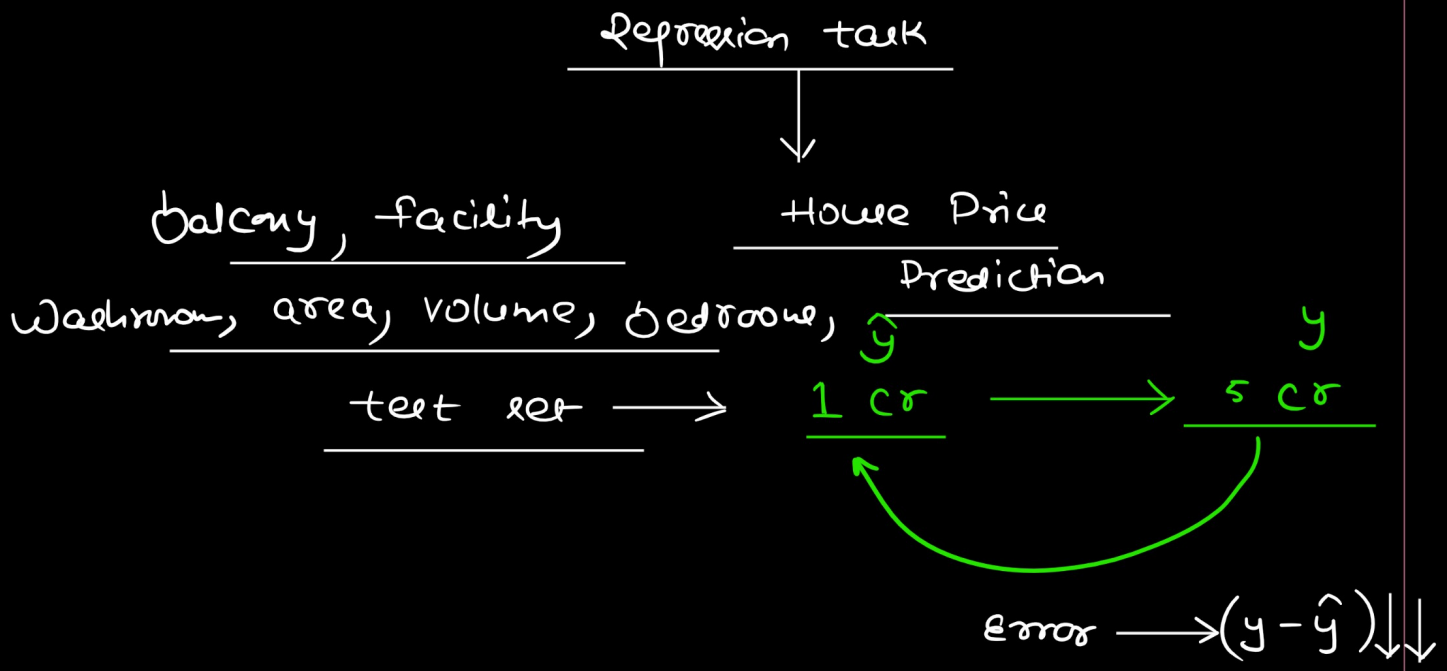
Limitation of DL

- ① Difficult to visualize the internals of model in DL
- ② cost requirement is quite huge
(GPUs)

At the beginning of training, these parameters are random. So the model's predictions are usually poor.

Through training:

The model makes a prediction. $y - \hat{y}$
It checks how wrong the prediction is. $\rightarrow$ Loss function (error) $\downarrow\downarrow$
It adjusts the parameters slightly. $\rightarrow$
It tries again with new data. $\rightarrow$
Over time, the adjustments accumulate, and the model gets better. $\leftarrow$



Epoch, Batch size & Iteration

10,000 samples

An epoch is one complete pass through the entire training dataset.

batch size = 100 samples

$$1 \text{ epoch} \Rightarrow \frac{10000}{100} = 100 \text{ iterations}$$

Instead of showing all data at once, we show the model small portions of data called batches.

batch size

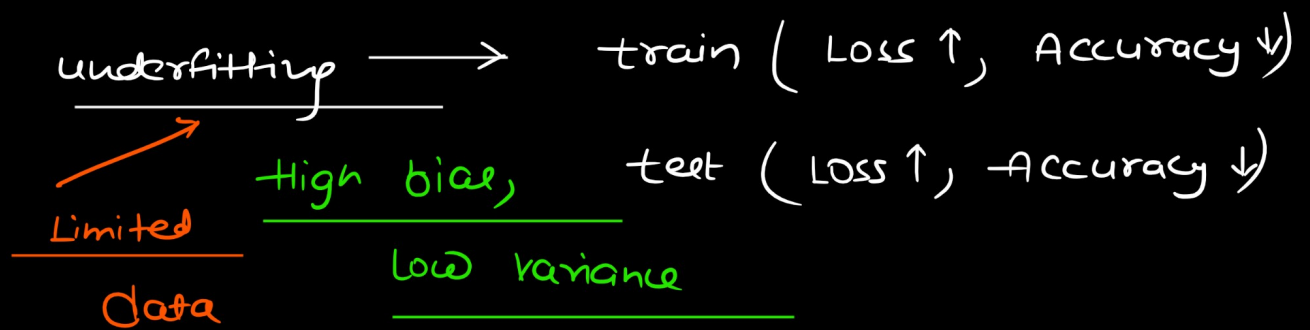
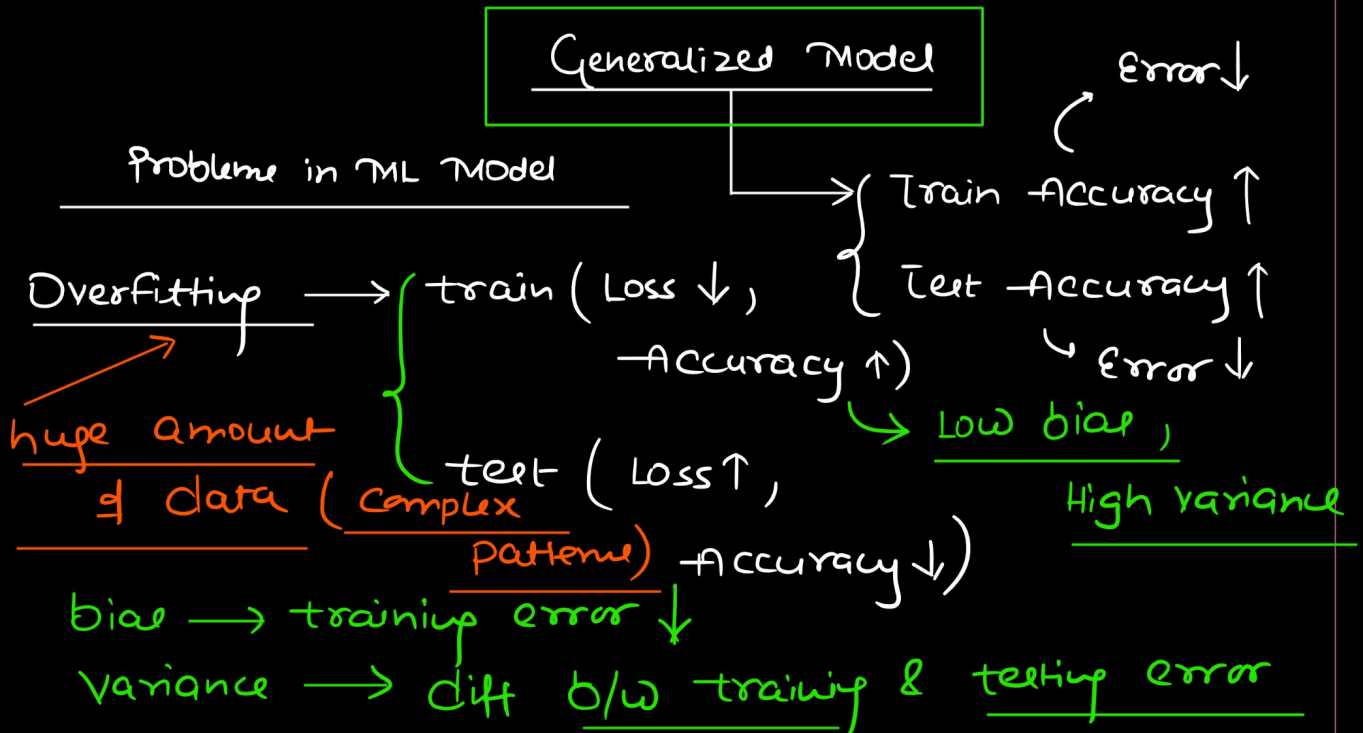
(batches)

An iteration is a single update step based on one batch of data.

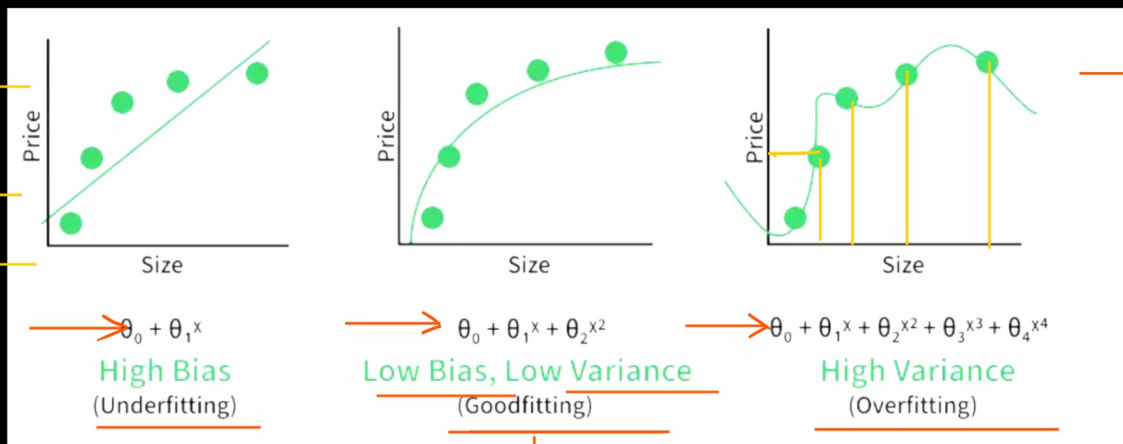
So:

Iterations per epoch = number of batches per epoch

If you have 10,000 rows and batch size is 100 → 100 iterations per epoch



training error
quite
huge



training error
 ≈ 0

Generalized Model $\rightarrow$ Low bias
Low
Variance

Underfitting

Underfitting happens when the model is too simple and cannot capture the actual patterns in the data.

Symptoms:

poor accuracy on training data

poor accuracy on test data

predictions feel "too basic" or incomplete

Example: Trying to fit a straight line to data that clearly forms a curve.

Real-life analogy:

A student who studied very little and fails both practice and final exams.

Overfitting

Overfitting happens when the model memorizes the training data too well. It performs extremely well on the data it has seen but poorly on new examples.

Symptoms:

very high training accuracy

low test accuracy

model reacts strongly to tiny details or noise

Example: A student who memorizes past test papers word-for-word but fails when given slightly different questions.

What We Really Want: Generalization

A good model:

learns real patterns

ignores noise

performs well on unseen data

Generalization is the goal of all training — it means the model has truly learned, not memorized.

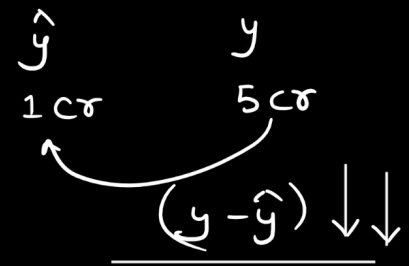
1. Training Set — "Learning Phase"

The training set is the portion of data the model directly uses to learn patterns.

During training:

the model sees the inputs,
compares predictions with actual outputs,
adjusts its parameters based on the error,
and repeats this process many times.

This is like a student practicing with many exercises to understand the topic.



A good training set must be:

- large enough
- clean
- diverse
- representative of the real world

because the model builds its entire understanding from this portion.

2. Validation Set — "Tuning & Decision Phase"

The validation set acts like a mock exam. It contains data the model has not seen during training.

We use it to:

- check how well the model is improving
- choose between different models
- tune hyperparameters (settings like learning rate, depth, layers)
- detect overfitting early
- decide when to stop training

The validation set helps us answer questions like:

Has the model learned too much (overfitting)?

Is the model learning too little (underfitting)?

Which model performs best?

Should we change settings to reduce error?

It is never used for final evaluation—only for making decisions during development.



3. Test Set — "Final Performance Check"

The test set is kept untouched until the very end. It serves as the final exam for the model.

We use it to measure:

how well the model will perform in real-world situations

whether the model generalizes correctly

whether we can trust the model beyond the training environment

If the model performs well on the training set and validation set but poorly on the test set, it means:

our validation process was flawed, or

the model has overfitted to both training and validation data.

This is why the test set must remain isolated until the development is complete.

Model Evaluation

gap b/w y & $\hat{y}$

classification
tasks

→ accuracy, precision,
recall, f1-score

Confusion
matrix

regression
tasks

→ $MSE \rightarrow \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
 $RMSE \rightarrow \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$

Clustering
tasks

→ silhouette
score

Deep → Dice Score, IOU
Learning task

GenAI → BLUE, ROUGE

Classification

← Confusion Matrix

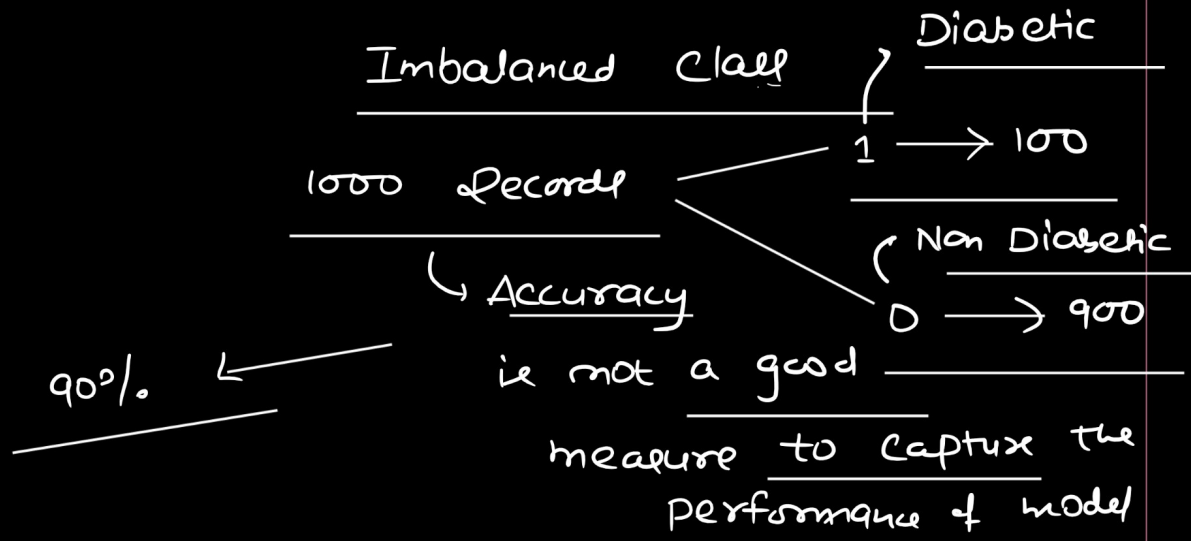
		Predicted	
		0	1
Actual	0	TN ↑	FP ↓
	1	FN ↓	TP ↑

Non Diabetic
Diabetic

$$\text{Accuracy} \rightarrow \frac{TN + TP}{TN + FP + FN + TP}$$

$$\text{Recall} \rightarrow \frac{TP}{TP + FN}$$

$$\text{Precision} \rightarrow \frac{TP}{TP + FP}$$



xx

Monitoring the performance

→ Data Science